Aperture — Low-Level Design

Module: Authentication

Version: v1.0

Module Owner: Identity & Access

Dependencies:

- Users Module
- Database
- Email Service (future)
- Notification Service (future)

---

1. Overview

The Authentication module is responsible for establishing and maintaining a user's identity within Aperture.

Its responsibilities are intentionally limited to:

- Authentication
- Session management
- Identity verification
- Token lifecycle
- Password management
- OAuth integration

This module **does not** own user profile information (bio, avatar, preferences, etc.). Those belong to the Users module.

Keeping these concerns separate reduces coupling and makes future authentication changes less disruptive.

---

2. Responsibilities

The Authentication module is responsible for:

- User registration
- User login
- User logout

- Token generation
- Token refresh
- Password hashing
- Password reset
- Email verification (future)
- OAuth authentication
- Session invalidation
- Route authentication middleware

It is **not** responsible for:

- User profile editing
- User preferences
- Followers/following
- Roles beyond authentication concerns
- Recommendation personalization

---

3. Functional Requirements

The module shall support:

Registration

- Create new account
- Validate email uniqueness
- Securely hash password
- Create corresponding User record

Login

- Email/password authentication
- Google OAuth
- Future provider extensibility

Logout

- Invalidate refresh token
- Clear client session
- Prevent token reuse

Session Management

- Short-lived access tokens
- Long-lived refresh tokens
- Secure token rotation

Password Management

- Password hashing
- Password reset request
- Password reset confirmation

---

4. Module Boundaries

Inputs:

- Login request
- Registration request
- Refresh token
- OAuth callback

Outputs:

- Access token
- Refresh token
- Authenticated identity
- Authentication errors

---

5. High-Level Workflow

Registration

↓

Validate Request

↓

Check Existing User

↓

Hash Password

↓

Create Identity

↓

Create User Profile

↓

Issue Tokens

↓

Return Auth Response

Login

↓

Validate Credentials

↓

Verify Password

↓

Issue Tokens

↓

Return Session

Token Refresh

↓

Validate Refresh Token

↓

Rotate Refresh Token

↓

Generate New Access Token

↓

Return Updated Session

---

6. API Surface

The authentication module exposes endpoints for:

Authentication

- Register
- Login
- Logout
- Refresh Token

OAuth

- Google Login
- OAuth Callback

Password

- Forgot Password
- Reset Password

Identity

- Current User
- Verify Authentication Status

Exact endpoint definitions are documented in the API Specification document.

---

7. Data Model

Primary entities:

Authentication Identity

Stores:

- email
- password hash
- provider
- account status
- timestamps

Refresh Session

Stores:

- refresh token identifier
- expiration
- device metadata (future)
- revocation status

User Profile is owned by the Users module.

---

8. Security Design

Passwords

- Never stored in plaintext
- Strong hashing algorithm
- Salted automatically

Tokens

- Signed
- Short expiration
- Refresh rotation
- Revocation support

OAuth

- State validation
- Secure callback handling

Sensitive operations require re-authentication where appropriate.

---

9. Validation Rules

Registration:

- Valid email
- Unique email
- Password complexity
- Username rules (if collected)

Login:

- Existing account
- Correct password
- Active account

Refresh:

- Valid refresh token
- Not revoked
- Not expired

---

10. Error Handling

Representative error categories:

Authentication

- Invalid credentials
- Account not found
- Account disabled

Registration

- Email already exists
- Invalid password
- Invalid request

Token

- Expired token
- Invalid signature
- Revoked refresh token

OAuth

- Provider unavailable
- Invalid callback
- Authorization cancelled

Errors should expose only information that is safe for clients to receive.

---

11. Database Interactions

Reads:

- Lookup identity by email
- Lookup refresh session

Writes:

- Create identity
- Store refresh session
- Revoke session
- Update password hash

Transactions should ensure identity creation and associated user creation remain consistent.

---

12. Dependencies

Authentication depends on:

Users Module

Creates a corresponding user profile during registration.

Database

Persists authentication records.

Future Email Module

Password reset Email verification

Future Notification Module

Security alerts New login notifications

Dependencies should remain one-directional.

---

13. Caching Strategy

Authentication data should rarely be cached.

Possible cache candidates:

- public signing keys (if applicable)
- rate-limiting counters
- revoked token lookups (if required)

Avoid caching sensitive identity information.

---

14. Performance Considerations

Authentication traffic is typically bursty.

Optimizations include:

- indexed email lookups
- efficient password verification
- lightweight token validation
- minimal database queries per request

Authentication endpoints should prioritize correctness over raw throughput.

---

15. Testing Strategy

Unit Tests

- Password hashing
- Token generation
- Validation logic

Integration Tests

- Registration flow
- Login flow
- Refresh flow
- Logout flow

API Tests

- Authentication endpoints
- Authorization middleware

Security Tests

- Invalid credentials
- Token tampering
- Expired tokens
- Replay attempts

---

16. Future Evolution

Potential enhancements:

- Multi-factor authentication
- Apple OAuth
- GitHub OAuth
- Device management
- Login history
- Trusted devices
- Security dashboard
- Passkeys (WebAuthn)

These should extend the module without requiring changes to other domains.

---

17. Interview Discussion Points

Be prepared to explain:

- Why JWT plus refresh tokens instead of server-side sessions?
- Why separate Authentication from User Profiles?
- How are passwords protected?
- How does refresh token rotation improve security?
- How would you revoke sessions across multiple devices?
- How would this design evolve to support MFA and passkeys?

---

18. Summary

The Authentication module is intentionally narrow in scope. It owns identity and session lifecycle while delegating user profile concerns to the Users module.

This separation improves maintainability, simplifies future enhancements, and aligns with the broader modular architecture of Aperture.